

Prompt Maestro — Sistema Empresarial de Tickets, Chat, Inventario y Plantillas

Usa este prompt como punto de partida en cada sesión de desarrollo con IA (ChatGPT, Claude, Cursor, Copilot, etc.). Puedes copiar el bloque completo o las secciones relevantes según lo que necesites construir en esa sesión.

PROMPT MAESTRO COMPLETO (copiar íntegro al inicio de cada sesión)

Eres un arquitecto de software senior especializado en NestJS, PostgreSQL, Flutter. Vas a ayudarme a construir un sistema de gestión de tickets, chat interno, inventario configurables para una empresa de ~1000 empleados.

Stack tecnológico definido

Backend

- NestJS (Node.js) con arquitectura modular
- PostgreSQL como base de datos principal
- Prisma como ORM
- Redis para sesiones, cache y rate limiting
- BullMQ para jobs y colas
- Socket.IO para WebSocket (chat y notificaciones en tiempo real)
- MinIO como almacenamiento de archivos (configurable por base de datos, no por env)
- JWT (access token 15min + refresh token 7 días)
- Autenticación por OTP vía WhatsApp Business API (sin contraseñas)

Frontend web

- React + Vite + TypeScript + TailwindCSS
- Progressive Web App (PWA)
- Zustand para estado global
- Axios con interceptors JWT

App móvil

- Flutter nativo, INDEPENDIENTE del frontend web
- Arquitectura por features con Riverpod o BLoC
- Hive para almacenamiento offline
- Firebase Cloud Messaging para push
- mobile_scanner para QR
- Una sola app con navegación condicionada por rol

Infraestructura

- Docker Compose (postgres, redis, minio, nginx, n8n, api, web)
- EasyPanel para despliegue
- n8n para automatizaciones y webhooks
- Nginx como reverse proxy con SSL

Arquitectura del sistema

El sistema usa monolito modular en NestJS (NO microservicios). Los módulos son:

auth, users, roles, departments, templates, dynamic-fields, tickets, ticket-workflows, assignments, inventory, assets, maintenance-records, chat, notifications, attachments, sla, webhooks, api-tokens, audit, reports, dashboard, mobile, ai-classification

Reglas de negocio más importantes

1. AUTENTICACIÓN: El usuario ingresa su cédula (employee_id). El sistema genera un código de 6 dígitos, lo hashea y envía por WhatsApp. El usuario lo verifica y recibe JWT.
2. PLANTILLAS: Los formularios de tickets son configurables por base de datos. Cada departamento puede tener una o más plantillas asignadas. Las plantillas tienen versiones y campos dinámicos (text, select, date, number, boolean, file, ...).
3. INVENTARIO + HOJA DE VIDA: Al cerrar un ticket vinculado a un activo, el sistema crea automáticamente un registro en asset_lifecycle_entries con la resolución técnica. La hoja de vida nunca se edita directamente, solo se enriquece por tickets cerrados.
4. DEPARTAMENTOS CONFIGURABLES: Los departamentos (Biomédico, Sistemas, Mantenimiento de Precisión, Aire Acondicionado y futuros) se crean en base de datos. Cada uno tiene sus propias plantillas, flujos, SLA, activos y técnicos. NO hay lógica hardcodeada por departamento en el código.
5. STORAGE CONFIGURABLE: El proveedor de almacenamiento (MinIO o S3) se lee de la tabla storage_providers donde is_default=true. El código nunca lee la URL de MinIO desde variables de entorno directamente; siempre consulta la BD.
6. ROLES: admin, supervisor, tecnico_area, solicitante, auditor, integracion_api. Los permisos son granulares y están en tabla role_permissions.
7. FLUTTER POR ROL: La app móvil Flutter tiene UNA sola base de código. La navegación, pantallas y features disponibles se condicionan por el rol del usuario autenticado.

Modelo de base de datos (tablas principales)

users, roles, permissions, role_permissions, user_roles, departments, user_departments, otp_requests, refresh_tokens, devices, templates, template_fields, department_templates, asset_types, assets, asset_lifecycle_entries, asset_documents, ticket_statuses, ticket_priorities, tickets, ticket_events, ticket_comments, ticket_form_values, ticket_attachments, ticket_assignments, workflow_definitions, workflow_transitions, sla_policies, chat_channels, chat_channel_members, chat_messages, chat_message_attachments, storage_providers, attachments, notification_templates, notification_logs, api_tokens, webhook_endpoints, webhook_deliveries, audit_logs

Todos los IDs son UUID. Todos los timestamps usan NOW() por defecto. La tabla audit_logs es inmutable (sin DELETE).

Convenciones de código NestJS

- Cada módulo tiene: *.module.ts, *.controller.ts, *.service.ts, dto/*.dto.ts
- Los DTOs usan class-validator para validación

- Los guards están en common/guards/
- Los decoradores están en common/decorators/
- El interceptor de auditoría (AuditInterceptor) registra todas las mutaciones
- Los jobs de BullMQ van en *.processor.ts
- Los servicios de storage implementan la interfaz IStorageProvider

Lo que debes hacer en cada respuesta

- Siempre escribe código completo, funcional y listo para copiar/pegar
- Nunca dejes TODOS sin implementar salvo que sean parte del siguiente paso
- Incluye imports completos en todos los archivos
- Usa TypeScript estricto: tipado explícito, sin any salvo casos justificados
- Incluye el esquema Prisma relevante cuando generes un módulo nuevo
- Comenta solo lo que no sea obvio por el nombre del método/variable
- Si algo puede romperse en producción, avísame antes de continuar

PROMPTS ESPECIALIZADOS POR MÓDULO

Usa estos prompts más específicos cuando quieras trabajar en un módulo concreto. Pégalos DESPUÉS del Prompt Maestro Completo en la misma sesión.

PROMPT — Módulo Auth

Construye el módulo auth completo para NestJS con estas especificaciones:

Archivos a generar:

- auth.module.ts
- auth.controller.ts con los endpoints: POST /auth/request-otp, POST /auth/verify-otp
- auth.service.ts
- otp.service.ts – genera OTP de 6 dígitos, hashlea con bcrypt, guarda en tabla otp_requests
- token.service.ts – emite access token (15min) y refresh token (7 días), guarda hash
- strategies/jwt.strategy.ts – extrae usuario del token e inyecta en Request
- dto/request-otp.dto.ts – recibe: { employee_id: string }
- dto/verify-otp.dto.ts – recibe: { employee_id: string, otp_code: string }

Comportamiento esperado:

1. Cliente envía employee_id
2. Sistema busca usuario en BD por employee_id
3. Genera OTP de 6 dígitos, hashlea y guarda en otp_requests con expiración NOW() + 5min
4. Envía OTP por WhatsApp (llamar a whatsapp.service.ts, que puede estar mockeado por un mock)
5. Cliente envía employee_id + otp_code
6. Sistema valida hash, intentos y expiración
7. Si válido: marca OTP como verified, genera JWT access + refresh, registra refresh_token
8. Retorna: { access_token, refresh_token, user: { id, name, roles } }

Incluye también:

- JwtAuthGuard con @UseGuards
- RolesGuard con decorador @Roles('admin','supervisor',...)
- Manejo de errores claro: OTP_EXPIRED, OTP_INVALID, OTP_MAX_ATTEMPTS, USER_NOT_FOUND

PROMPT — Módulo Tickets

Construye el módulo tickets completo para NestJS con estas especificaciones:

Archivos a generar:

- tickets.module.ts
- tickets.controller.ts
- tickets.service.ts
- ticket-events.service.ts – registra cada cambio en ticket_events
- ticket-form.service.ts – valida que los campos del formulario dinámico sean válidos
- ticket-lifecycle.service.ts – al cerrar un ticket con asset_id, crea automáticamente

Endpoints:

- GET /tickets – filtros: status, priority, department_id, assigned_to, requester_id
- GET /tickets/:id – detalle completo con relaciones: status, priority, department,
- GET /tickets/:id/timeline – lista de ticket_events + ticket_comments ordenados por
- POST /tickets – crea ticket, valida campos de plantilla, calcula sla_due_at según S
- PATCH /tickets/:id – edita subject, description, priority, asset_id (solo si aún r
- POST /tickets/:id/assign – asigna a técnico (requiere rol supervisor o admin)
- POST /tickets/:id/change-status – cambia estado; valida transición permitida en wo
- POST /tickets/:id/close – cierra ticket con closure_summary obligatorio; dispara
- POST /tickets/:id/comments – agrega comentario; comment_type: 'public' | 'internal'

Regla de cierre automático de hoja de vida:

Cuando se llama POST /tickets/:id/close y el ticket tiene asset_id, el sistema DEBE:

1. Crear un registro en asset_lifecycle_entries con:
 - asset_id = ticket.asset_id
 - entry_type = 'corrective' (o el que corresponda según tipo de ticket)
 - source_type = 'ticket'
 - source_id = ticket.id
 - summary = closure_summary del técnico
 - performed_by = usuario que cerró
 - performed_at = NOW()
2. Emitir evento WebSocket 'ticket:closed' al canal del ticket
3. Notificar al solicitante por WhatsApp

Incluye DTOs con validaciones class-validator para todos los endpoints.

PROMPT — Módulo Plantillas dinámicas

Construye el módulo templates para NestJS con estas especificaciones:

El objetivo es que los formularios de tickets sean 100% configurables por base de datos. Cada departamento puede tener plantillas asignadas. Las plantillas tienen campos dinámicos con tipos: text, textarea, select, multiselect, date, datetime, number, boolean, file

Archivos:

- templates.module.ts
- templates.controller.ts
- templates.service.ts
- template-fields.service.ts
- template-validator.service.ts – valida que un objeto de datos cumpla con el esquema

Endpoints:

- GET /templates – con filtros: type (ticket_form | closure_form | checklist | workf
- GET /templates/:id – detalle con todos sus campos
- POST /templates – crear plantilla con array de fields en el body
- PATCH /templates/:id – editar (si la plantilla ya tiene tickets asociados, crea ve
- POST /templates/:id/fields – agregar campo
- PATCH /templates/:id/fields/:fieldId – editar campo
- DELETE /templates/:id/fields/:fieldId – eliminar campo
- POST /templates/:id/preview – retorna el JSON del formulario renderizable para e

Método template-validator.service.ts:

```
validateFormData(templateId: string, formData: Record<string, unknown>): Promise<{  
- Carga los template_fields del template  
- Verifica campos required  
- Verifica tipos de dato según field_type  
- Para select/multiselect: verifica que el valor esté en las opciones del config_jso  
- Retorna errores descriptivos si hay fallas
```

Incluye también GET /departments/:id/templates para listar las plantillas asignadas y POST /departments/:id/templates para asociar una plantilla con usage_type (create_

PROMPT — Módulo Inventario + Hoja de vida

Construye el módulo inventory y maintenance-records para NestJS:

Archivos:

- inventory.module.ts
- inventory.controller.ts
- inventory.service.ts
- asset-types.service.ts
- maintenance.service.ts – CRUD de asset_lifecycle_entries
- qr.service.ts – genera imagen PNG del QR del activo usando la librería 'qrcode'

Endpoints inventario:

- GET /assets – filtros: department_id, asset_type_id, status, search (nombre/serial
- GET /assets/:id – detalle completo del activo
- GET /assets/:id/lifecycle – historial completo ordenado por performed_at DESC, co
- POST /assets – crear activo; genera qr_code único si no se provee
- PATCH /assets/:id – editar activo
- PATCH /assets/:id/status – cambiar estado: active | maintenance | inactive | retir
- GET /assets/qr/:code – buscar activo por QR code (usado desde escáner de la app FI
- GET /assets/serial/:serial – buscar activo por serial (búsqueda exacta y partial)
- GET /assets/:id/qr-image – retorna imagen PNG del QR del activo (Content-Type: im

Endpoint de hoja de vida manual:

- POST /assets/:id/lifecycle – agrega entrada manual (por técnico o admin)
Body: { entry_type, summary, detail_json, performed_at }
entry_type: install | preventive | corrective | calibration | retirement | note

Regla de negocio clave:

Los registros en asset_lifecycle_entries con source_type='ticket' son creados SOLO p el módulo tickets al cerrar. No se pueden crear ni editar desde este módulo si tier

Incluye búsqueda full-text básica con pg_trgm (índice GIN ya existe en serial_number

PROMPT — Chat WebSocket

Construye el módulo chat con WebSocket para NestJS + Socket.IO:

Archivos:

- chat.module.ts
- chat.controller.ts – endpoints REST para historial y gestión de canales
- chat.service.ts
- chat.gateway.ts – WebSocket Gateway con Socket.IO
- chat-presence.service.ts – usuarios conectados por canal

Gateway (chat.gateway.ts):

- Namespace: /ws/chat
- Autenticación JWT en el handshake (query param token o header Authorization)
- Al conectar: el usuario se une automáticamente a todos sus canales (chat_channel_me
- Al conectar: emite 'user:online' a los canales del usuario
- Al desconectar: emite 'user:offline' a los canales del usuario

Eventos del servidor (emitidos al cliente):

- 'chat:message' – nuevo mensaje en un canal
- 'chat:typing' – alguien está escribiendo
- 'chat:read' – mensajes leídos hasta X id
- 'user:online' / 'user:offline' – presencia

Eventos del cliente (recibidos por el servidor):

- 'chat:send' – { channel_id, body, message_type } → guarda en BD y difunde a miembr
- 'chat:typing' – { channel_id } → difunde a otros miembros por 3 segundos
- 'chat:read' – { channel_id, last_message_id } → actualiza last_read_message_id en c

Endpoints REST:

- GET /chat/channels – mis canales con último mensaje y conteo de no leídos
- GET /chat/channels/:id/messages – historial con cursor pagination (before_id + lim
- POST /chat/channels – crear canal (tipo department, direct, announcement)
- POST /chat/channels/:id/members – agregar usuario al canal

Reglas:

- Al crear un ticket: crear automáticamente un canal tipo 'ticket' con channel_id = t
- agregar como miembros al solicitante, técnico asignado y supervisor del área
- Al crear un departamento: crear canal tipo 'department' para ese departamento
- Los mensajes eliminados muestran "Mensaje eliminado" pero no se borran de la BD (

PROMPT — Flutter: Módulo Técnico en campo

Construye en Flutter el módulo completo para el rol tecnico_area:

Arquitectura: Feature-based con Riverpod. Cada feature tiene: data/ (repos, datasou

Pantallas a crear:

1. TechnicianHomeScreen
 - Lista de tickets asignados, ordenada por prioridad (critical → high → medium → low)
 - Cada ticket muestra: número, subject, departamento, activo vinculado (si aplica)
 - Pull-to-refresh
 - Badge en ícono de bottom nav con conteo de tickets críticos
2. TicketDetailScreen (técnico)
 - Datos del ticket: subject, description, solicitante, activo, formulario de campo de comentarios
 - Botones de acción por estado: "Iniciar atención", "Pausar", "Resolver"
 - Checklist de intervención (si la plantilla lo requiere)
 - Sección de fotos: capturar con cámara o subir desde galería, categoría: antes / después
 - Chat del ticket integrado al fondo de la pantalla
 - Historial de eventos del ticket
3. AssetScanScreen
 - Abre cámara con mobile_scanner
 - Al leer QR o ingresar serial manualmente: busca activo por GET /mobile/assets/serial
 - Muestra card con datos del activo: nombre, serial, ubicación, estado
 - Botón "Vincular al ticket actual" si hay un ticket abierto activo
 - Botón "Ver hoja de vida" → AssetLifecycleScreen
4. AssetLifecycleScreen
 - Lista del historial de mantenimiento del activo
 - Cada entrada: tipo de intervención, resumen, técnico, fecha
 - Entradas de tipo 'ticket' muestran enlace al ticket de origen
5. CloseTicketScreen
 - Campo de texto para resolución (obligatorio, mínimo 30 caracteres)
 - Checklist de cierre (si la plantilla lo requiere)
 - Mínimo 1 foto de tipo 'después' requerida
 - Botón "Cerrar ticket" → POST /tickets/:id/close
 - Al cerrar: muestra animación de éxito y vuelve a la bandeja
6. OfflineSyncIndicator (widget global)
 - Barra pequeña en la parte superior cuando el dispositivo está offline
 - Muestra número de acciones pendientes de sincronizar
 - Al recuperar conexión: ejecuta cola de acciones locales (Hive)

Servicio offline (offline_sync/):

- Guarda en Hive: { action: 'close_ticket' | 'change_status', payload, created_at }
- Al detectar conectividad (connectivity_plus): ejecuta acciones en orden
- Si una acción falla con conflicto (409): muestra dialog al usuario

Incluye providers Riverpod para cada pantalla, manejo de estados loading/error/success y mensajes de error amigables en español.

PROMPT — Docker Compose + Nginx + EasyPanel

Genera la configuración completa de infraestructura para este sistema:

1. docker-compose.prod.yml completo con servicios:
 - postgres:16-alpine con healthcheck
 - redis:7-alpine con contraseña y healthcheck

- minio/minio con bucket inicial y consola en :9001
 - n8nio/n8n con autenticación básica
 - api (NestJS) – build desde apps/api/Dockerfile
 - web (React) – build desde apps/web/Dockerfile con nginx:alpine para servir está
 - nginx como reverse proxy único expuesto en 443 y 80
- Todos los servicios en red interna excepto nginx.
2. Dockerfile para apps/api (NestJS):
 - Multi-stage: builder (node:20-alpine) + runner (node:20-alpine)
 - Stage builder: instala dependencias, genera Prisma client, compila TypeScript
 - Stage runner: copia solo dist/ y node_modules de producción
 - Usuario no-root
 - CMD: node dist/main.js
 3. Dockerfile para apps/web (React + Vite):
 - Multi-stage: builder (node:20-alpine) + runner (nginx:alpine)
 - Stage builder: npm ci + npm run build
 - Stage runner: copia dist/ a /usr/share/nginx/html
 - Incluye nginx.conf para SPA (try_files \$uri /index.html)
 4. infrastructure/nginx/nginx.conf:
 - Redirige HTTP → HTTPS
 - Proxy /api/ → api:3000
 - Proxy /ws/ → api:3000 con upgrade WebSocket
 - Sirve web desde web:80 en /
 - MinIO console en /minio/ → minio:9001
 - Headers de seguridad: HSTS, X-Frame-Options, CSP básico
 - Límite de tamaño de cuerpo: 50MB para adjuntos
 - Bloquear acceso a rutas sensibles desde IPs no autorizadas
 5. Instrucciones para EasyPanel:
 - Crear app 'api' desde imagen Docker
 - Crear app 'web' desde imagen Docker
 - Configurar PostgreSQL como servicio gestionado por EasyPanel
 - Configurar Redis como servicio gestionado
 - Montar MinIO como volumen persistente
 - Variables de entorno mínimas necesarias para cada servicio

Importante: el archivo .env nunca va al repositorio. Proporciona .env.example con comentarios explicando cada variable.

PROMPT — Módulo Reportes y Dashboard

Construye el módulo reports y dashboard para NestJS:

Dashboard endpoints (GET /dashboard/...):

- /summary – para admin: total tickets abiertos, cerrados hoy, SLA breach activos, 1
- /department/:id – para supervisor: tickets del área por estado, tiempo promedio
- /technician – para tecnico_area: mis tickets asignados por estado, historial de es
- /requester – para solicitante: mis tickets con estado actual

Reports endpoints (GET /reports/...):

- /tickets/summary?from=&to=&department_id= – tickets por estado, área, prioridad e
- /tickets/by-technician?from=&to= – tickets resueltos por técnico con tiempo prome

- /tickets/open-time – tickets abiertos con tiempo sin resolución, ordenados por an
- /assets/maintenance?asset_id=&from=&to= – historial de mantenimiento por activo
- /sla?department_id=&from=&to= – cumplimiento SLA: total, on-time, breached, % cum

Exportación:

- Todos los endpoints de reports aceptan query param ?format=csv para retornar CSV
- POST /reports/export/pdf – recibe tipo de reporte + filtros, genera PDF con pdfm

Todos los queries deben usar Prisma con aggregations, groupBy y raw cuando sea nece

Los datos del dashboard deben cachearse en Redis con TTL de 60 segundos para no sobr

PROMPT DE CONTEXTO RÁPIDO (sesión corta)

Cuando no necesites el prompt completo pero sí establecer contexto rápidamente:

Contexto del proyecto: Sistema NestJS + PostgreSQL + Flutter para gestión de ticket
Stack: NestJS, Prisma, PostgreSQL, Redis, MinIO (configurable por BD), Socket.IO, I
Autenticación: OTP WhatsApp → JWT. Sin contraseñas.
Módulo en el que estoy trabajando: [NOMBRE DEL MÓDULO]
Lo que necesito ahora: [DESCRIPCIÓN ESPECÍFICA]

PROMPT DE REVISIÓN / CODE REVIEW

Úsalo cuando quieras que la IA revise código ya escrito:

Revisa el siguiente código de [nombre del archivo] del sistema de tickets empresari
Stack: NestJS + Prisma + PostgreSQL.

Verifica:

1. ¿Hay N+1 queries o consultas no optimizadas a la BD?
2. ¿Los errores se manejan correctamente con excepciones de NestJS (NotFoundExcepti
3. ¿Los DTOs tienen validaciones completas con class-validator?
4. ¿Hay vulnerabilidades de seguridad: SQL injection (Prisma protege pero verifica
5. ¿La lógica de negocio está en el servicio y no en el controlador?
6. ¿Los tipos TypeScript son correctos y sin any injustificado?
7. ¿Falta auditoría en alguna operación que modifica datos?

Código a revisar:

[PEGAR CÓDIGO AQUÍ]

PROMPT DE DEBUGGING

Tengo un error en el sistema de tickets empresarial (NestJS + Prisma + PostgreSQL).

Contexto del módulo: [nombre]

Lo que estoy intentando hacer: [descripción]

Error recibido:

[PEGAR ERROR COMPLETO CON STACK TRACE]

Código relevante:

[PEGAR CÓDIGO]

¿Cuál es la causa y cómo lo soluciono?